

not gate

Truth tables :

a	not a
0	1
1	0

a	a	a nand a
0	0	1
1	1	0

So : $\text{not}(a) = a \text{ nand } a$

And in hdl : `Nand(a=in, b=in, out=out);`

and gate

Since $a \text{ nand } b = \text{not}(a \text{ and } b)$, $a \text{ and } b = \text{not}(a \text{ nand } b)$. So $a \text{ and } b = (a \text{ nand } b) \text{ nand } (a \text{ nand } b)$ based on the not gate.

In hdl, the result of $a \text{ nand } b$ is stored into $n1$ and then we compute $n1 \text{ nand } n1$:

```
Nand(a=a, b=b, out=n1);  
Nand(a=n1, b=n1, out=out);
```

or gate

Truth tables :

a	b	a or b	not(a)	not(b)	not(a) and not(b)	not(not(a) and not(b))
0	0	0	1	1	1	0
0	1	1	1	0	0	1
1	0	1	0	1	0	1
1	1	1	0	0	0	1

So : $a \text{ or } b = \text{not}(\text{not}(a) \text{ and } \text{not}(b)) = \text{not}(a) \text{ nand } \text{not}(b)$

In hdl :

```
Nand(a=a, b=a, out=nota);  
Nand(a=b, b=b, out=notb);  
Nand(a=nota, b=notb, out=out);
```

xor gate

The `xor.hdl` file contains 2 versions. The first one uses only `Nand` gates. The second one (commented in the code) uses `Not`, `And` and `Or` gates. The translation can be seen through the following truth table :

a	b	not(a)	not(b)	not(a) and b	a and not(b)	(not(a) and b) or (a and not(b))
0	0	1	1	0	0	0
0	1	1	0	1	0	1
1	0	0	1	0	1	1
1	1	0	0	0	0	0

And also through the fact that $a \text{ xor } b$ can be seen as $(a \text{ or } b) \text{ and } \text{not}(a \text{ and } b)$. Which can be expressed as $(a \text{ and } \text{not}(b)) \text{ or } (b \text{ and } \text{not}(a))$. I don't know if there is a simpler have or or a more elegant way to express the gate.

mux gate

From the definition of the gate, $\text{mux}(a, b, \text{sel}) = (a \text{ and } \text{not}(\text{sel})) \text{ or } (b \text{ and } \text{sel})$ which can be translated as $(a \text{ nand } (\text{sel} \text{ nand } \text{sel})) \text{ nand } (b \text{ nand } \text{sel})$. And can be verified through the following truth table :

a	b	sel	sel nand sel	a nand (sel nand sel)	b nand sel	(a nand (sel nand sel)) nand (b nand sel)
0	0	0	1	1	1	0
0	1	0	1	1	1	0
1	0	0	1	0	1	1
1	1	0	1	0	1	1
0	0	1	0	1	1	0
0	1	1	0	1	0	1
1	0	1	0	1	1	0
1	1	1	0	1	0	1

to be continued...